Free Prompt Pack

A free sample from the Browser Game Starter Kit by Prompt Forge Kits: one real genre prompt from the kit's Platformer sequence, plus a 30-minute walkthrough for getting your first game running with an AI coding tool.

Before you start (5 minutes)

You need three things: Node.js 18+ (LTS), Git, and an AI coding tool that opens a local folder (Cursor, Claude Code, Windsurf, or VS Code + Copilot). That's it. The game is drawn in code, so you don't need any art files.

The 4 rules that make vibe-coded games actually ship

1. **One prompt = one feature.** Never ask for five things at once.
2. **Test after every change.** Play it. Open the browser console (F12) and look for red errors.
3. **Commit every working step.** Git is your save game.
4. **Three failed fixes = roll back.** Then ask for a smaller step.

First game in 30 minutes

Minutes 0-5: scaffold. In a terminal, create a Vite project and add Phaser 3:

```
npm create vite@latest my-first-game -- --template vanilla
cd my-first-game
npm install
npm install phaser
git init && git add -A && git commit -m "empty project"
```

Open the `my-first-game` folder in your AI tool.

Minutes 5-10: give the AI its rules, then build a tiny platformer. Paste this as your first message:

```
You are helping me build a small browser game with Phaser 3 and Vite. Rules:
- One feature per step. Never change more than I ask for.
- Keep it small: a single scene is fine for now, split files only when a file
passes ~200 lines.
- Put every tunable number (speed, gravity, jump power) in a config.js file as
named constants.
- Draw all graphics in code with Phaser Graphics (rectangles, circles). No image
files.
- After each change, tell me exactly how to test it in 1-2 sentences.
- If I paste an error, explain the cause in one sentence before you fix it.

Step 1: replace the starter content with a Phaser 3 game, 800x450, arcade physics,
gravity 900. Draw a cyan circle player, an olive ground strip along the bottom,
and 3 floating slate platforms. Arrow keys move left/right, Space jumps (only when
standing on something). Don't add anything else.
```

Minutes 10-15: run it and commit.

```
npm run dev
```

Open the link Vite prints. You should see a circle you can move and jump. If it works, save your progress:

```
git add -A && git commit -m "step 1: player moves and jumps"
```

Minutes 15-25: add the free genre prompt below. It's a real prompt from the kit's Platformer sequence, and it's the one that makes a platformer feel good to play.

Minutes 25-30: add one more feature of your own, test, commit. Pick something small: a coin you can collect, a spike that restarts the level, a score in the corner. Write it as one prompt, with a "how do I test it" line at the end.

Your free genre prompt: Better jump feel (Platformer)

Paste this after step 1 works. It adds the two tricks nearly every good platformer uses.

```
Improve jump feel in GameScene: add coyote time (100 ms after leaving a ledge you
can still jump) and jump buffering (pressing jump up to 100 ms before landing
still jumps). Put both values in config.js.
```

Test: Run off a ledge and press jump slightly late. You still jump. Press jump a moment before you land. You jump as soon as you touch down.

If your scene isn't called `GameScene`, just tell the AI the right name. Commit when it passes: `git add -A && git commit -m "jump feel"`.

When something breaks: the Fix Loop

AI tools are fast at writing code and also fast at breaking it. The habit that keeps you shipping:

```
Change breaks something
  -> Try 1: paste the exact error + what you did
  -> Try 2: give more context: the file contents, what you expected
  -> Try 3: ask the AI to explain the cause BEFORE fixing
  -> Still broken? ROLL BACK to your last commit. Re-ask with a smaller step.
```

To roll back to your last save point:

```
git checkout .     # restore changed files
git clean -fd      # remove new untracked files (careful: deletes them)
```

Rolling back costs two minutes. Spiraling costs an evening.

Make every game look like yours: the "Chunky Flat Toy" look

The kit gives all six genres one consistent style, so a game built from it looks deliberate instead of random. Four rules you can use right now in any game:

- **Thick dark outline** (3 px, color `#1b1f2e`) around every character and pickup. Backgrounds get no outline.
- **Flat shapes, one highlight each:** a small white blob at about 50% alpha near the top-left. No gradients on characters.
- **Color means something:** cyan `#3ec9e6` is the player, red `#ef4f4f` is danger, gold `#ffc93c` is money and rewards, green `#5bd16b` is healing, purple `#a45ee5` is magic or rare.
- **Muted backgrounds, bright actors:** use soft slate and olive tones behind, so your characters always stand out.

Paste those four rules into your AI tool's rules file and your next game inherits the look.

What's next

You now have a running game, a save-point habit, and a recovery plan. The full **Browser Game Starter Kit** ($29, one-time) is the next step: a finished Phaser 3 + Vite starter game, rules files your AI tool reads automatically, six genre sequences with 60+ tested prompts (platformer, top-down roguelite, idle/clicker, endless runner, puzzle, tower defense), the full Fix Loop guide, and a publishing guide. It comes with a 7-day refund and free updates. Details: promptforgekits.com. Store pages are still launching, so there's no rush and no pressure.

Free Prompt Pack v1 by Prompt Forge Kits, promptforgekits.com. For your personal use; please don't redistribute.